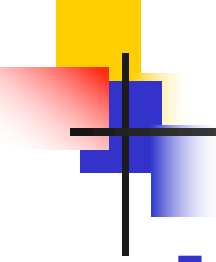
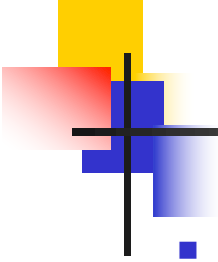


Tema 6: Gestión de Estado con Redux



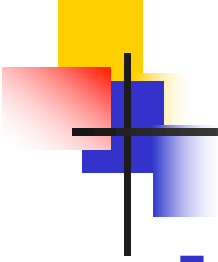
Índice

- Motivación
- Introducción a Redux
- Desarrollo iterativo de un sencillo gestor de TODOs
- Estado normalizado



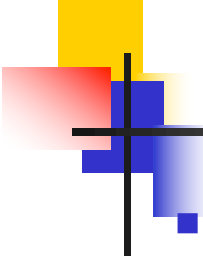
Motivación

- En pa-todos el componente **App** contiene
 - Todo el estado de la aplicación
 - La lógica de modificación del estado ante todas las posibles acciones del usuario
- En general, esta aproximación en una aplicación real no sería viable
 - Gran parte del estado tendría que pasarse mediante propiedades desde **App** a componentes hijo, y así sucesivamente hacia abajo
 - Sería inmanejable y causaría excesivas renderizaciones en el Virtual DOM
 - Además, **App** contendría una cantidad de lógica excesiva
- Una segunda aproximación podría ser
 - Distribuir el estado (y la lógica de modificación) entre algunos componentes “contenedores”
 - Los componentes contenedores delegan en otros componentes (componentes “de presentación”) contenidos, directa o indirectamente, en ellos
 - Los componentes de presentación reciben parte del estado como propiedades y callbacks para manipular el estado ante las acciones del usuario (como hace **App** con **AddTodo**, **Todos [Todo]** y **Filter**)



Motivación

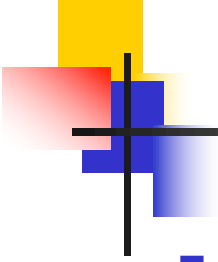
- Sin embargo, esta segunda aproximación no es realista
 - Porque se está asumiendo que el estado que gestiona un componente contenedor sólo necesita ser visto (mediante propiedades) desde los componentes de presentación que están contenidos, directa o indirectamente, en él
- Ejemplo: PA-Shop
 - Los datos de perfil de usuario se guardan como estado (caché)
 - **Header** muestra el **userName** del usuario
 - **UpdateProfile** muestra gran parte de los datos del perfil y causa su modificación
 - **Login** y **SignUp** causan su inicialización
 - ¿Qué componente padre, común sólo a ellos, podría guardar (y gestionar) en su estado los datos del perfil?



Motivación

Ejemplo: PA-Shop

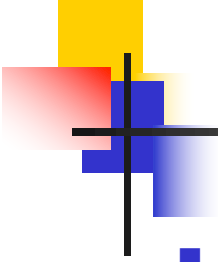
- Los datos del carrito se guardan como estado (caché)
 - `ProductDetails` permite añadir un producto al carrito
 - `ShoppingCart` muestra los datos del carrito y permite modificarlos
 - `Buy` muestra los datos del carrito y lo vacía
 - `ShoppingCartIcon` (en la cabecera) muestra el número de ítems en el carrito
 - ¿Qué componente padre, común sólo a ellos, podría guardar (y gestionar) en su estado los datos del carrito?
- Los datos resumen de los productos que muestra la página actual del resultado de una búsqueda se guardan como estado (caché)
 - `FindProducts` lanza la búsqueda inicial
 - `FindProductsResult` muestra el resultado y permite navegar a anterior/siguiente
 - `ProductDetails` que permite volver a la “página anterior” (`FindProductsResult`), sin tener que volver a hacer una petición al backend
 - ¿Qué componente padre, común sólo a ellos, podría guardar (y gestionar) en su estado los datos del resultado de una búsqueda de productos?



Motivación

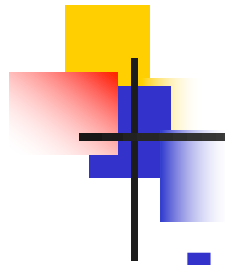
- Ejemplo: PA-Shop

- Pasa lo mismo con **FindOrders**, **FindOrdersResult** y **OrderDetails**
- La información sobre todas las categorías (id y nombre) se guarda como estado (caché)
 - En las búsquedas de productos y la información de detalle de un producto el backend no incluye el nombre de la categoría, sino sólo su id
 - **FindProductsResult** y **ProductDetails** recuperan el nombre de la categoría del estado a partir del id
 - **FindProducts** muestra todas las categorías en el desplegable
 - ¿Qué componente padre, común sólo a ellos, podría guardar (y gestionar) en su estado los datos de las categorías?
- Etc.



Introducción a Redux

- Redux (<https://redux.js.org>) es una librería que implementa una **arquitectura funcional** para gestionar **estado global**
 - **Estado global:** estado que se establece y recupera desde varios componentes
- Permite gestionar el estado global de una aplicación de forma **centralizada** y **modular**
- No está ligado a React y puede usarse con otros frameworks
- Nosotros usaremos react-redux (<https://react-redux.js.org>)
- Los componentes pueden recibir notificaciones cuando el estado global cambia
- El estado que sólo es relevante a un componente lo seguiremos modelando como **estado local** (hook `useState`)
 - **Estado local:** estado que se establece y recupera desde un único componente



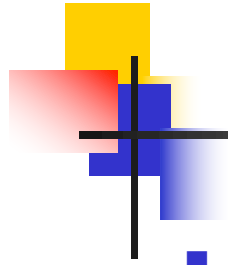
Conceptos principales

■ **store**

- Es un objeto de Redux que almacena el estado global de la aplicación
- El estado tiene propiedades (e.g. `todos`, `filterType`)

■ **Acciones**

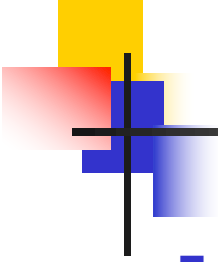
- Cada vez que se quiere modificar el estado global (normalmente como consecuencia de una interacción del usuario), hay que enviar un objeto acción a **store**
 - `store.dispatch(action)`
- La acción debe contener la propiedad **type** para indicar su tipo (`type: 'ADD_TODO'`) y puede contener propiedades adicionales para los datos (e.g. `text: 'Learn Redux'`)



Conceptos principales

■ Reductor

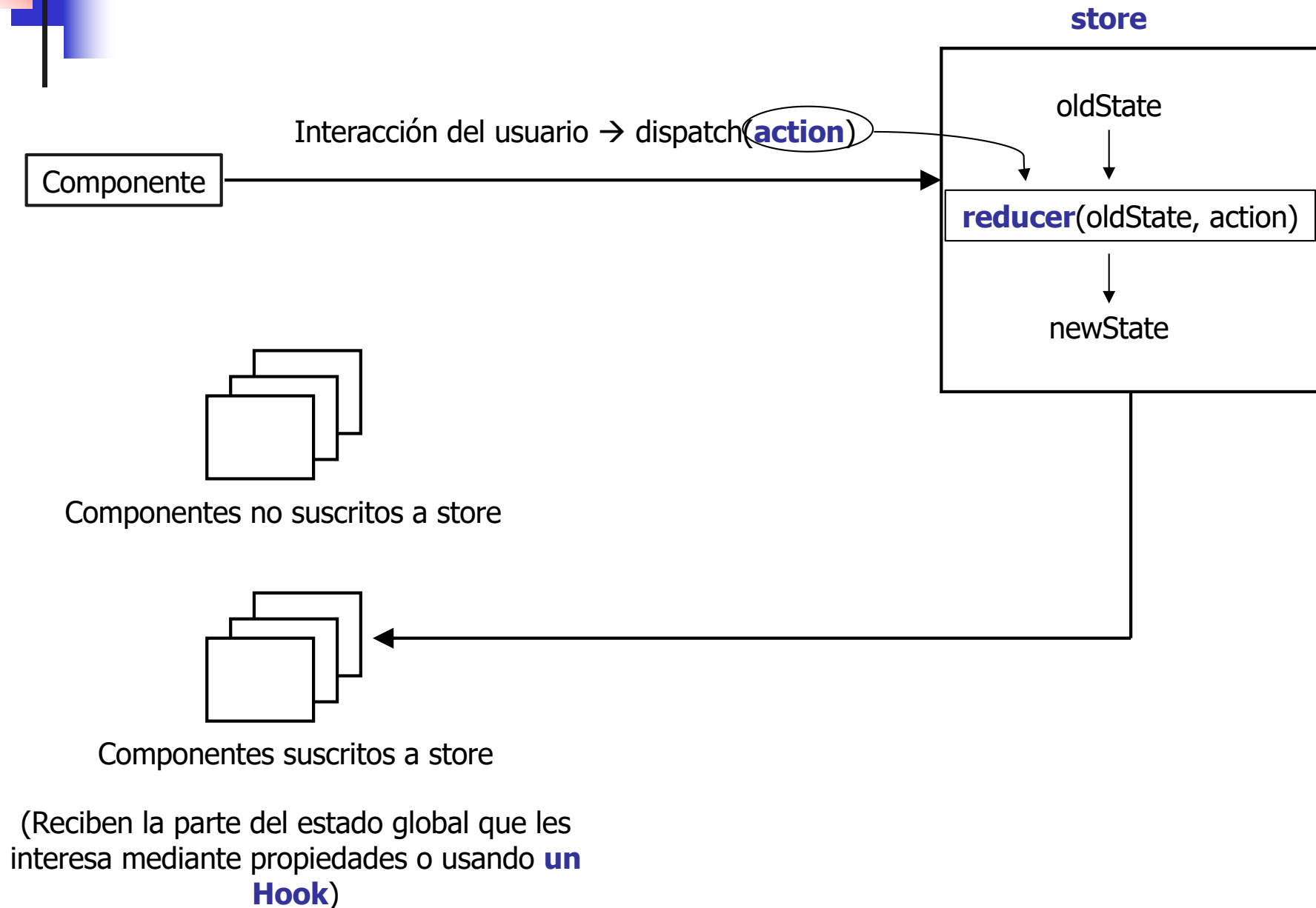
- Es una función encargada de producir un nuevo estado ante una acción
- Se ejecuta cuando se invoca a `store.dispatch(action)`
- Devuelve el nuevo estado a partir de la acción y el estado anterior
- **Debe ser una función pura**: con los mismos parámetros siempre devuelve el mismo resultado
- **El estado debe tratarse como inmutable**: un reductor no modifica el estado anterior, sino que devuelve un nuevo objeto
- Las ventajas de la inmutabilidad se estudiaron en el tema anterior
- En este tema observaremos cómo Redux reduce el número de renderizaciones en el Virtual DOM gracias a esta propiedad



Conceptos principales

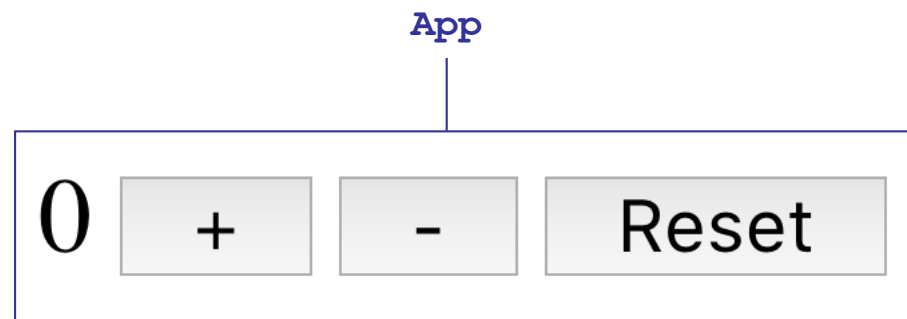
- Los componentes que lo deseen pueden recibir una notificación cada vez que el reductor produce un nuevo estado
 - Se pueden suscribir con `store.subscribe` y leer el estado con `store.getState`
 - Los componentes pueden recibir la parte del estado global que les interesa mediante propiedades o usando **un Hook**
 - Haciendo la analogía con el patrón Observer en diseño orientado a objetos, el `store` es un Subject y los componentes suscritos son Observers

Flujo



Un primer ejemplo

- `git clone git@github.com:udc-fic-pa/pa-counter-redux.git`
- `cd pa-counter-redux`
- `npm install`
- `npm run dev`
- **Objetivo: reimplementar pa-counter con Redux**



src/main.jsx

```
...  
import {configureStore} from '@reduxjs/toolkit';  
import {Provider} from 'react-redux';
```

```
...  
import App from './App';  
import reducer from './reducer';
```

```
/* Configure store. */
```

```
const store = configureStore({reducer});
```

Creación del `store` (ver siguiente diapositiva)

```
/* Render application. */
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>
```

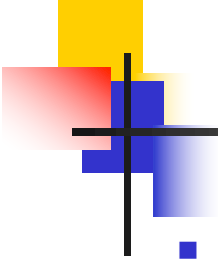
```
    <Provider store={store}>
```

```
      <App/>
```

```
    </Provider>
```

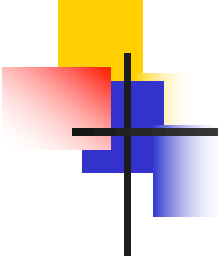
```
  </React.StrictMode>);
```

- El componente `Provider` permite pasar el `store` a todos los componentes en el árbol de componentes (no lo reciben como una propiedad, sino a través del mecanismo Context de React)
- El uso de este componente es necesario para poder suscribir componentes a `store` (`store.subscribe`)



Creación del store

- Para la creación del objeto **store** usamos la función **configureStore** de la librería Redux Toolkit
 - <https://redux-toolkit.js.org>
- **configureStore** permite crear el objeto **store** y añade un conjunto de capacidades adicionales sobre Redux
 - Integración con el plugin Redux DevTools para navegadores (más adelante)
 - [Modo desarrollo] Detección de modificaciones del estado
- **configureStore** recibe un objeto que puede tener varios campos
 - El campo **reducer** especifica el reductor
 - `const store = configureStore({reducer: reducer});`
- En JavaScript moderno no es necesario especificar el valor de una propiedad si se inicializa con una variable del mismo nombre
 - `const store = configureStore({reducer});`



App.jsx

```
import {useSelector, useDispatch} from 'react-redux';
```

```
import * as actions from './actions';
```

```
export const getValue = state => state.value;
```

```
import * as selectors from './selectors';
```

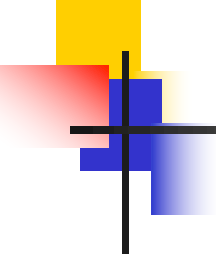
```
const App = () => {
```

```
  const value = useSelector(selectors.getValue);
```

---> Hook: suscripción

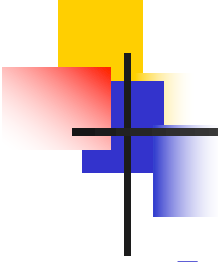
```
  const dispatch = useDispatch();
```

---> Hook: obtención de la función
dispatch ligada a store



App.jsx

```
    return (  
      <div>  
        {value + ' '}  
        <button onClick={() => dispatch(actions.increment())}>+</button>  
        {' '  
        <button onClick={() => dispatch(actions.decrement())}>-</button>  
        {' '  
        <button onClick={() => dispatch(actions.reset())}>Reset</button>  
      </div>  
    );  
  }  
  
  export default App;
```



useSelector

- **Suscribe** el componente al store
- Recibe un **selector** como parámetro
 - En terminología Redux, un selector es una función pura que devuelve un valor a partir del estado almacenado en el store
 - Los selectores también permiten ocultar la estructura interna del estado a los componentes
 - La estructura interna del estado sólo se conoce en `reducer.js` y `selectors.js`
 - Si se refactoriza la estructura interna del estado, los componentes suscritos no se verán afectados
- Cuando se ejecuta una acción, cada componente suscrito al store se vuelve a renderizar si alguno de los selectores devuelve un valor distinto al de la última vez
 - **Para la comparación se usa `===` (igualdad referencial)**
 - Esta estrategia asume que el estado es inmutable (recuérdese que un reductor debe tratar el estado como inmutable)

actions.js

```
export const increment = () => ({type: 'INCREMENT'});  
export const decrement = () => ({type: 'DECREMENT'});  
export const reset = () => ({type: 'RESET'});
```



- Estas funciones encapsulan la creación de las acciones
- En terminología Redux se llaman “action creators”
- Simplifican y hacen más mantenible el código de los componentes

reducer.js

```
const reducer = (state={value: 0}, action) => {
```

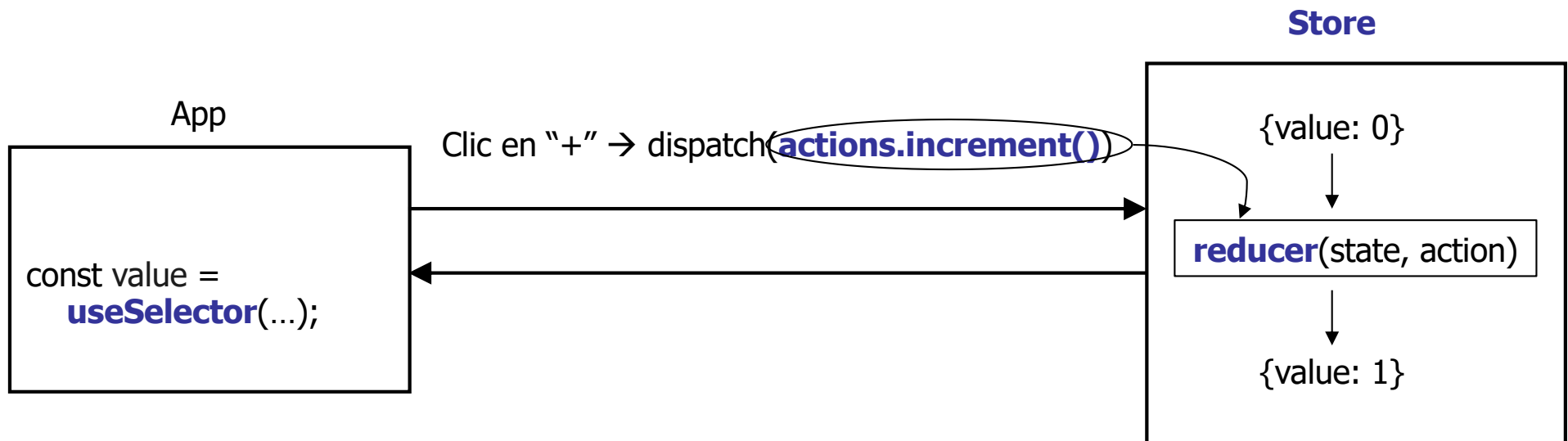
```
  switch(action.type) {
    case 'INCREMENT':
      return {value: state.value+1};
    case 'DECREMENT':
      return {value: state.value-1};
    case 'RESET':
      return {value: 0};
    default:
      return state;
  }
```

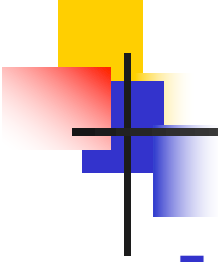
```
}
```

```
export default reducer;
```

- Para obtener el estado inicial, Redux invoca al reductor con (`undefined`, `{'type': <<valor aleatorio único>>}`)
- El reductor debe devolver el estado inicial
- El idioma habitual para devolver el estado inicial consiste en usar la sintaxis de argumentos por defecto en el parámetro `state` e incluir un caso `default` para devolverlo

Flujo



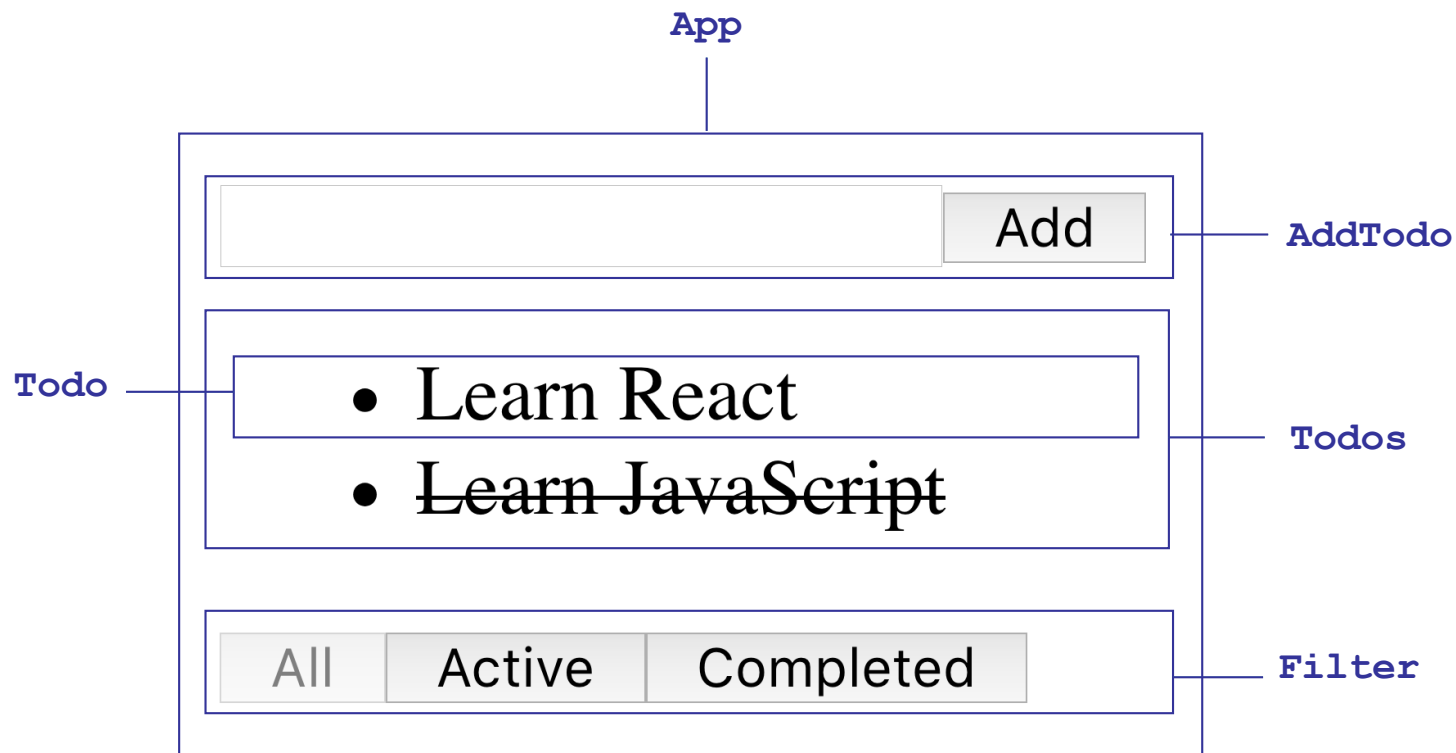


Redux DevTools

- Para visualizar gráficamente las acciones y el estado global se puede instalar el plugin Redux DevTools en el navegador
 - Como se comentó anteriormente, la función `configureStore` realiza la integración con el plugin si éste está instalado
 - Instalación del plugin en el navegador
 - <https://github.com/reduxjs/redux-devtools/tree/main/extension#installation>
 - Sólo hay que seguir las instrucciones de instalación del plugin en el navegador (`configureStore` se encarga de integrar Redux con el plugin automáticamente)
 - Todos los ejemplos de la asignatura, así como el esqueleto de la práctica, usan `configureStore`
 - Pestaña Redux en las herramientas del navegador
- Demo

Desarrollo iterativo de un sencillo gestor de TODOs

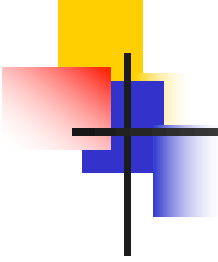
- Reimplementaremos pa-todos con Redux de forma iterativa



it-1

- `git clone git@github.com:udc-fic-pa/pa-todos-redux.git`
- `cd pa-todos-redux`
- `git checkout it-1`
- `npm install`
- `npm run dev`
- **Objetivo: crear una primera versión que permita añadir TODOs y cambiar su estado**



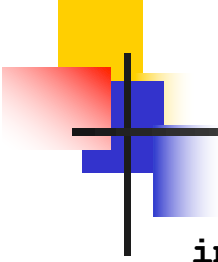


[it-1] App.jsx

```
import AddTodo from './AddTodo';  
import Todos from './Todos';
```

```
const App = () => (  
  <div>  
    <AddTodo/>  
    <Todos/>  
  </div>  
);
```

```
export default App;
```



[it-1] AddTodo.jsx

```
import {useDispatch} from 'react-redux';

import * as actions from './actions';

const AddTodo = () => {

  const dispatch = useDispatch();
  let input;

  return (
    <form onSubmit={ (e) => {
      ...
      dispatch(actions.addTodo(input.value.trim()));
      ...
    }}>
      ...
    </form>
  );
}

export AddTodo;
```

[it-1] Todos.jsx

```
import {useSelector, useDispatch} from 'react-redux';
```

```
import Todo from './Todo';
```

```
import * as actions from './actions';
```

```
import * as selectors from './selectors';
```

```
const Todos = () => {
```

```
  const todos = useSelector(selectors.getVisibleTodos);
```

```
  const dispatch = useDispatch();
```

```
  return (
```

```
    <ul>
```

```
      {todos.map(todo =>
```

```
        <Todo key={todo.id} todo={todo}
```

```
          onToggleCompleted={id =>
```

```
            dispatch(actions.toggleTodo(id)) }/>)
```

```
      }
```

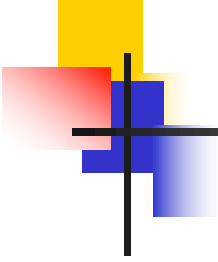
```
    </ul>
```

```
  );
```

```
}
```

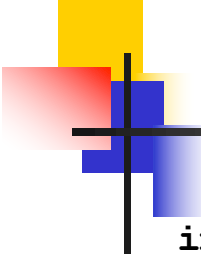
```
export default Todos;
```

```
export const getVisibleTodos =  
  state => state.todos;
```



[it-1] actionTypes.js

```
export const ADD_TODO = 'ADD_TODO';  
export const TOGGLE_TODO = 'TOGGLE_TODO';
```



[it-1] actions.js

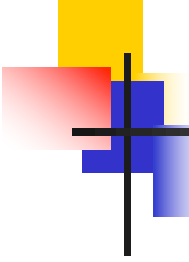
```
import * as actionTypes from './actionTypes';
```

```
let nextTodoId = 0;
```

```
const todo = text => ({  
  id: nextTodoId++,  
  text,  
  completed: false,  
});
```

```
export const addTodo = text => ({  
  type: actionTypes.ADD_TODO,  
  todo: todo(text)  
});
```

```
export const toggleTodo = id => ({  
  type: actionTypes.TOGGLE_TODO,  
  id  
});
```



[it-1] reducer.js

```
import * as actionTypes from './actionTypes';

const initialState = {
  todos: []
};

export const reducer = (state=initialState, action) => {

  switch(action.type) {
    case actionTypes.ADD_TODO:
      return {todos: [action.todo, ...state.todos]};

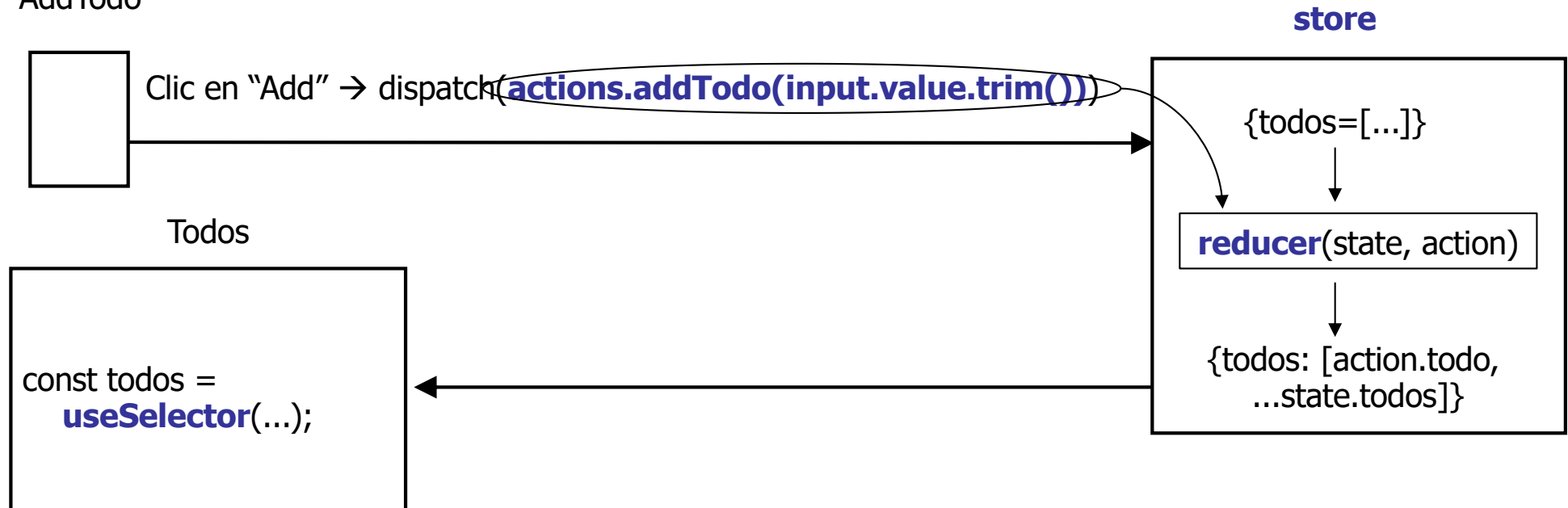
    case actionTypes.TOGGLE_TODO:
      return {todos: state.todos.map(todo => {
        return todo.id === action.id ?
          {...todo, completed: !todo.completed} : todo)}}

    default:
      return state;
  }
}

export default reducer;
```

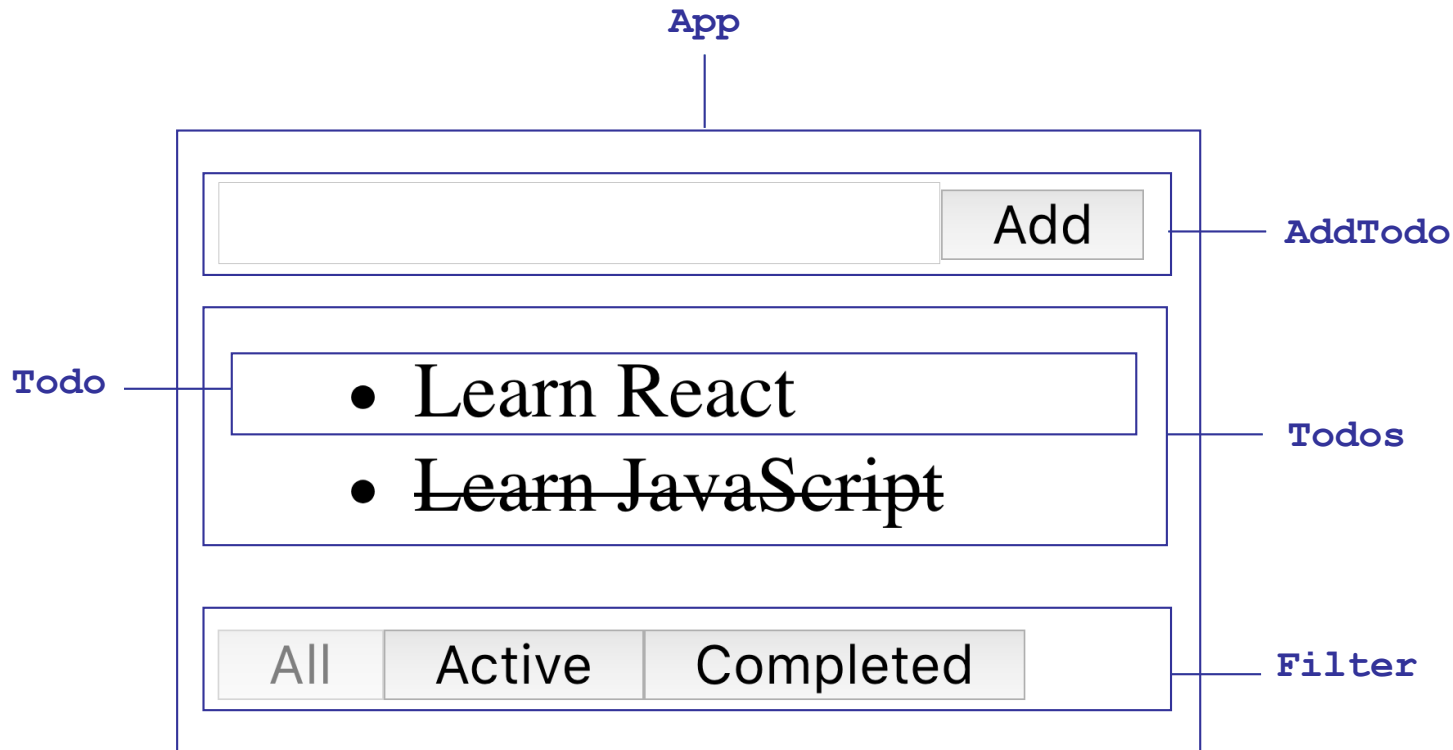
[it-1] Flujo

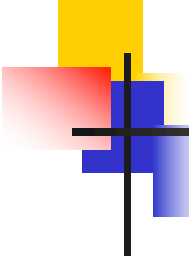
AddTodo



it-2

- `git checkout it-2`
- **Objetivo: añadir un filtro que permita mostrar todos los TODOs, sólo los activos o sólo los completados**





[it-2] Filter.jsx

```
import {useSelector, useDispatch} from 'react-redux';

import * as filterTypes from './filterTypes';
import * as actions from './actions';
import * as selectors from './selectors';

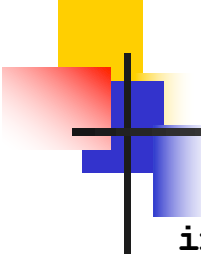
const Filter = () => {

  const filterType = useSelector(selectors.getFilterType);
  const dispatch = useDispatch();

  return (
    <div>
      <button disabled={filterType === filterTypes.ALL}
        onClick={() => dispatch(
          actions.setFilter(filterTypes.ALL) }>All</button>
      ...
    </div>
  );
}

export default Filter;
```

El estado global ahora tiene dos propiedades: `todos` y `filterType`



[it-2] selectors.js

```
import * as filterTypes from './filterTypes';

export const getVisibleTodos = state => {

  switch(state.filterType) {
    case filterTypes.ALL:
      return state.todos;
    case filterTypes.ACTIVE:
      return state.todos.filter(todo => !todo.completed);
    case filterTypes.COMPLETED:
      return state.todos.filter(todo => todo.completed);
    default:
      throw new Error(`Unknown filter type ${state.filterType}`);
  }

}

export const getFilterType = state => state.filterType;
```

[it-2] reducer.js

...

```
export const reducer = (state=initialState, action) => {  
  
  switch(action.type) {  
  
    case actionTypes.ADD_TODO:  
      return {...state, todos: [action.todo, ...state.todos]};  
  
    case actionTypes.TOGGLE_TODO:  
      return {...state, todos: state.todos.map(todo => {  
        return todo.id === action.id ?  
          {...todo, completed: !todo.completed} : todo)}}}  
  
    case actionTypes.SET_FILTER:  
      return {...state, filterType: action.filterType}  
  
    default:  
      return state;  
  }  
}
```

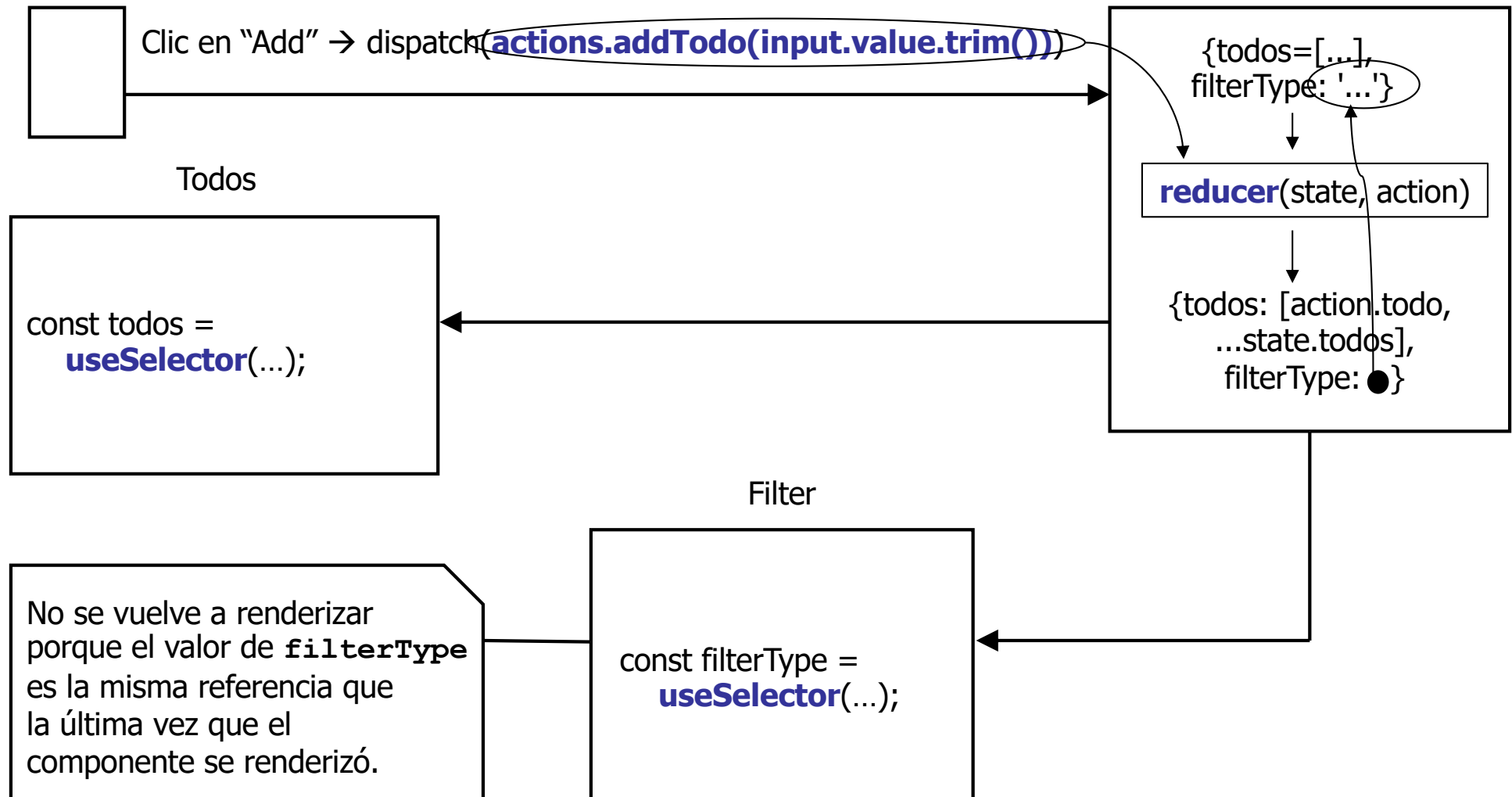
...

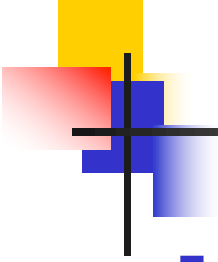
El operador de propagación evita tener que escribir explícitamente `filterType: state.filterType` en los dos primeros casos y `todos: state.todos` en el tercero

[it-2] Flujo

AddTodo

store

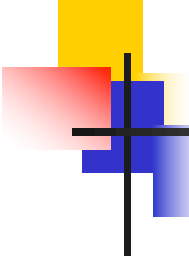




[it-2] Ejercicio

- ¿Qué componentes se vuelven a renderizar cuando se hace clic en un TODO?
- ¿Y cuando se selecciona un filtro?

- `git checkout master`
- **Objetivo: implementar el reductor como una composición de reductores más sencillos**
 - El reductor mostrado anteriormente trata las actualizaciones de `todos` y `filterType` de forma independiente
 - Las dos primeros casos tratan con `todos` y el tercero con `filterType`
 - Esta situación es muy común
 - En una aplicación más grande, para que el código sea más manejable, lo normal sería descomponer el reductor en reductores más pequeños, y finalmente crear un reductor que los combine
 - Cada reductor “pequeño” trata con una parte del estado global



[it-3] reducer.js

```
import {combineReducers} from 'redux';

...

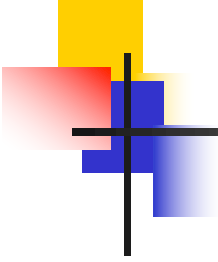
const initialState = {
  todos: [],
  filterType: filterTypes.ALL
};

const todos = (state=initialState.todos, action) => {

  switch(action.type) {
    case actionTypes.ADD_TODO:
      return [action.todo, ...state];

    case actionTypes.TOGGLE_TODO:
      return state.map(todo => {
        return todo.id === action.id ?
          {...todo, completed: !todo.completed} : todo));

    default:
      return state;
  }
};
```



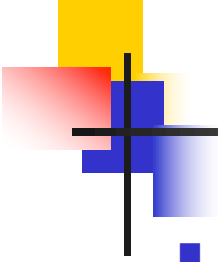
[it-3] reducer.js

```
const filterType = (state=initialState.filterType, action) => {  
  
  switch(action.type) {  
  
    case actionTypes.SET_FILTER:  
      return action.filterType;  
  
    default:  
      return state;  
  }  
  
};
```

```
const reducer = combineReducers({  
  todos,  
  filterType  
});
```

Se explica en la siguiente diapositiva

```
export default reducer;
```

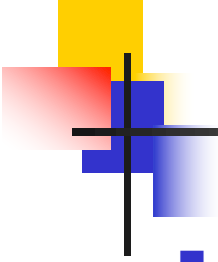


[it-3] combineReducers

- **combineReducers**

- **Recibe** un objeto $\{p_1: r_1, \dots, p_n: r_n\}$, siendo p_i propiedades y r_i funciones reductoras
- **Devuelve** una función reductora “normal”, es decir, una función con la firma $(state, action) \rightarrow state$, que realiza su trabajo de la siguiente manera
 - Cada reductor r_i es invocado con $state.p_i$ y la acción $action$
 - Devuelve el objeto $\{p_1: s_1, \dots, p_n: s_n\}$, donde s_i es el estado devuelto por el reductor r_i
- De nuevo, el código que se muestra en la diapositiva anterior usa la sintaxis abreviada de JavaScript moderno
 - No es necesario especificar el valor de una propiedad si se inicializa con una variable del mismo nombre
 - Sería equivalente a escribir

```
const reducer = combineReducers({
  todos: todos,
  filterType: filterType
});
```



[it-3] combineReducers

- El uso de **combineReducers** en **reducer.js** produciría un resultado equivalente a implementar **reducer** como

```
const reducer = (state={}, action) => ({  
  todos: todos(state.todos, action),  
  filterType: filterType(state.filterType, action)  
});
```